ALEPH CONSULTING - PRESENTATION COMPANION

OpenClaw: The Homelab Guide

From Tonight's Talk to Your First Agent

PRESENTATION

Companion guide for Braxton Heaps' presentation — March 19, 2026

ORGANIZATION

Aleph Consulting | aleph-consultants.com

Practical field handout for attendees setting up OpenClaw after the talk.

Warm, practical, grounded.
Built for one-sitting readability.

What You Just Saw

Tonight you watched a self-hosted AI agent running on real hardware — reading infrastructure, recalling decisions from weeks ago, coordinating across multiple instances, and printing physical labels on demand. None of that required a cloud service, a SaaS subscription, or anyone's permission.

That's OpenClaw. A self-hosted gateway that sits between AI models, your messaging channels, and your tools. It runs on your machine, connects to your infrastructure, and remembers what you tell it.

This guide is the path from what you just saw to running your own.

Table of Contents

What OpenClaw Actually Is 4

The Five Moving Parts 5

Pick Your Deployment Pattern 5

Install and Validate 6

Your Workspace — The Files That Matter 8

Connect a Channel 10

Remote Access Without Being Reckless 11

Security for People Who Run Infrastructure 12

Memory That Works 14

Skills — Teach It Your Infrastructure 15

Heartbeats and Cron — Your AI Ops Loop 17

Model and Cost Strategy 18

When You're Ready for More 19

Your First Week — Day by Day 21

Operations Runbook 23

Troubleshooting 24

Lessons from 48 Days in Production 25

Reference Commands 27

What's on This USB Drive 29

Resources 29

1 What OpenClaw Actually Is

OpenClaw is a self-hosted gateway for AI agents. It is not a chatbot wrapper. It is a control plane that connects:

- **AI model providers** (Anthropic, OpenAI, local Ollama models)
- **Messaging channels** (Matrix, Discord, Telegram, Signal, WhatsApp)
- **Your workspace** (Markdown files that define behavior, memory, and context)
- **Tools** (shell commands, web search, file operations, browser automation)
- **Nodes** (paired devices — phones, remote machines)

The gateway is the process that stays running. Everything else plugs into it.

It is free, open source, and runs on anything from a Raspberry Pi to a Proxmox cluster.

> **What makes it different from ChatGPT or Claude web:**

	ChatGPT/Claude Web	Codex/Claude Code	OpenClaw
Memory	Resets between sessions	Project-scoped, resets	Persists in files you control
Tools	Limited to provider	Code and terminal	Shell, files, web, browser, custom skills
Infrastructure access	None	Repo and terminal only	Full access to your network
Identity	Generic	Generic	Defined by your SOUL.md and workspace
Data	On their servers	On their servers	On your hardware
Customization	System prompts	Limited	Full workspace, skills, and plugins
Cost model	Subscription	Subscription	API keys — pay for what you use
Always-on	No	No	Yes — heartbeats, cron, monitoring
Messaging channels	No	No	Matrix, Discord, Telegram, Signal

2 The Five Moving Parts

If you remember one thing from this guide, make it this.

```
┌──────────────────────────────────────────────────┐
│                    GATEWAY                       │
│              (the always-on process)             │
│                                                  │
│  ┌──────────┐  ┌─────────┐  ┌──────────────┐    │
│  │ WORKSPACE │  │ MODELS  │  │    TOOLS     │    │
│  │ (files)   │  │ (brains) │  │   (hands)    │    │
│  └──────────┘  └─────────┘  └──────────────┘    │
│                                                  │
│  ┌──────────────────────────────────────────┐    │
│  │              CHANNELS                    │    │
│  │     (Matrix, Discord, Telegram, ...)     │    │
│  └──────────────────────────────────────────┘    │
└──────────────────────────────────────────────────┘
```

1. **Gateway** — The service. Start it, stop it, restart it. This is what you operate.
2. **Workspace** — Markdown files that define behavior, memory, and identity. Edit a file, change the agent.
3. **Models** — The LLMs. Claude, GPT, Ollama, whatever. These cost money (or run locally for free).
4. **Tools** — What the agent can actually do. Read files, run commands, search the web, hit APIs.
5. **Channels** — How messages reach the gateway. Your inbox.

When something breaks, ask which layer is broken. That habit alone will save you hours.

Minimum viable OpenClaw is one host, one model, one channel, one operator, and a healthy dashboard. What you saw in the presentation is a mature deployment — six weeks of investment across four agents, a GPU cluster, and vector memory. That is not where you start. It is where one path leads.

3 Pick Your Deployment Pattern

Decide where OpenClaw lives *before* you install it.

Pattern A: Your Workstation

BEST FOR: Learning, first install, daytime use.

Install on your laptop or desktop. Easiest start. Not ideal for 24/7 uptime, and mistakes happen on the machine you actually use.

Pattern B: Dedicated Homelab Box ← Recommended

BEST FOR: Serious daily use, safe growth.

A spare mini PC, an Ubuntu VM on Proxmox/ESXi, or any always-on Linux host. Clean separation from your daily driver. Easy to snapshot, back up, and SSH into.

This is what we run. Seven's Home instance runs on a $75 refurbished Touch Dynamic J6412. The Build Triad runs on a Proxmox VM.

Pattern C: VPS

BEST FOR: Remote-first use, fixed uptime.

Valid, but should be your *second* deployment. Easier to expose badly. Some sites dislike datacenter IPs.

The Right Sequence

1. Install locally or on a test VM
2. Learn the dashboard and workspace
3. Add one channel
4. Harden it
5. Move to dedicated always-on hardware

4 Install and Validate

Install

macOS / Linux / WSL2:

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Windows PowerShell:

```
iwr -useb https://openclaw.ai/install.ps1 | iex
```

> **Then run onboarding:**

```
openclaw onboard --install-daemon
```

The wizard sets up gateway auth, workspace defaults, model providers, and optionally a messaging channel. Use **QuickStart** unless you already know why you need Advanced.

What the Onboarding Wizard Is Actually Doing

This isn't cosmetic. The wizard configures:

- Gateway authentication (how the dashboard and API are secured)
- Local vs remote gateway binding
- Workspace directory and defaults
- Model provider credentials
- Optional channel connections
- Service installation (daemon mode)

Understanding this matters later when something breaks. Each of these is a potential failure point.

Validate Before You Do Anything Else

Your first job is not to personalize the bot. Your first job is to prove the stack is healthy.

```
openclaw gateway status    # Is the service running?
openclaw doctor            # Any critical problems?
openclaw dashboard         # Open the control UI
```

Send a test message in the dashboard. If you get a response, you have a working stack.

> **Do not add channels until this works. Do not add skills until this works. Do not add remote access until this works.**

The Five-Layer Diagnostic

When something breaks, ask: **which layer is broken?**

Layer	Symptom	First Check
Gateway	Nothing works at all	`openclaw gateway status`
Models	Bot is silent or errors	`openclaw models status`

Layer	Symptom	First Check
Workspace	Bot responds but ignores context	Check file paths and permissions
Channels	Dashboard works but messaging doesn't	Channel status in dashboard
Tools	Bot can't execute actions	Tool policy config, exec approvals

This habit will make you much faster at troubleshooting than "it's broken, help."

5 Your Workspace — The Files That Matter

The workspace lives at `~/.openclaw/workspace/` by default. These files shape everything about how your agent behaves. Edit them with any text editor. They're Markdown.

Core Files

File	Purpose	Loads Every Session?
`AGENTS.md`	Operating instructions — priorities, rules, workflow	Yes
`SOUL.md`	Tone, persona, boundaries, identity	Yes
`USER.md`	Facts about you — name, timezone, preferences	Yes
`IDENTITY.md`	The agent's own identity — name, voice, character	Yes
`TOOLS.md`	Local tool notes and conventions	Yes
`MEMORY.md`	Curated long-term memory	Yes
`HEARTBEAT.md`	Periodic check instructions	On heartbeat
`memory/YYYY-MM-DD.md`	Daily working notes	Searchable

THE KEY INSIGHT: every file that loads every session costs tokens every message. Keep `MEMORY.md` lean (under 3,000 characters). Move details to `memory/` subdirectory files — the agent searches those on demand.

SOUL.md — Who Your Agent Is

This is optional but transformative. Without it, you have a generic assistant. With it, you have *your* assistant.

A minimal SOUL.md:

```
# SOUL.md

You are Atlas, my homelab assistant.

## Tone
- Be direct and concise
- Don't apologize for things that aren't your fault
- When unsure, say so

## Boundaries
- Don't run destructive commands (rm -rf, DROP TABLE) without asking me first
- Don't expose internal IPs or credentials in responses
- If something could break production, warn me before proceeding

## Priorities
1. Don't break the NAS
2. Be helpful
3. Be honest about what you don't know
```

That's enough to start. You can grow it as the relationship develops.

AGENTS.md — How to Work

This is where you put operational instructions:

```
# AGENTS.md

## Memory
- Write important findings to memory/YYYY-MM-DD.md
- Keep MEMORY.md under 3,000 characters
- Promote durable facts from daily notes to MEMORY.md

## Working Style
- Check HEARTBEAT.md on each heartbeat
- When running commands, explain what you're doing before executing
- Prefer safe alternatives (trash > rm)
```

Workspace Is Not All of `~/.openclaw/`

Important distinction. Your workspace is the agent's working files. Outside the workspace, `~/.openclaw/` holds other things you should know about:

```
~/.openclaw/
├── workspace/          ← Your agent's files (Git this)
│   ├── SOUL.md
│   ├── AGENTS.md
│   ├── MEMORY.md
│   ├── memory/
│   └── skills/
├── config/             ← Gateway configuration
├── .env                ← API keys and secrets (DO NOT Git)
├── sessions/           ← Conversation history
├── skills/             ← Managed/shared skills
├── logs/               ← Gateway logs (chmod 600)
└── data/               ← Internal state
```

Back up the workspace with Git. Do not blindly Git the entire `~/.openclaw/` directory — it contains secrets, session history, and credentials.

```
cd ~/.openclaw/workspace
git init
git add -A
git commit -m "initial workspace"
```

Use a **private** Git repo. Your workspace contains your infrastructure details, preferences, and operational context. It's not sensitive like API keys, but it's not public information either.

6 Connect a Channel

Start with **one DM channel**. Not groups. Not multiple channels. One.

Why One?

- Easier to debug
- Easier to secure
- You'll know exactly where messages come from
- Less token burn from noise

Recommended First Channels

Matrix — Self-hosted option, E2EE, rooms for future multi-agent coordination. This is what we use. **Caveat:** it is a more opinionated setup path than Telegram for beginners, so choose it because you want Matrix, not because you want the easiest first channel.

Telegram — Easiest setup. Create a bot via @BotFather, add the token. Done.

Discord — Good if your community is already there. Create a bot application, add the token.

Signal — Strong privacy. Slightly more setup.

Channel Pairing and Access Control

This is **chat-user/channel pairing** — unknown senders get a short code, and you approve them deliberately:

```
openclaw pairing list <channel>
openclaw pairing approve <channel> <CODE>
```

FOR DAY ONE: pairing enabled, one DM channel, no groups.

Do not confuse this with device/node pairing. Approving a DM sender is a different flow from approving a phone, browser, or paired node.

Groups — Later

Groups add noise, cost, and prompt injection risk. When you're ready:

- Require @-mentions to activate
- Use sender allowlists
- Keep groups small and purposeful

7 Remote Access Without Being Reckless

The dashboard is an **admin surface**, not a harmless UI. If someone gets dashboard access, they can interact with your agent and, by extension, your infrastructure.

Safe Options (in order of preference)

1. Local browser — Simplest. Just run `openclaw dashboard`.

2. SSH tunnel:

```
ssh -N -L 18789:127.0.0.1:18789 user@gateway-host
```

Then open `http://127.0.0.1:18789/` locally.

3. Tailscale Serve — HTTPS within your tailnet. Stays private. Identity-aware.

4. Tailscale Funnel — Public HTTPS. This means publicly reachable. Use shared password at minimum. Treat like production exposure.

The Rule

Do not publicly expose the dashboard just because you can. Keep the gateway loopback-only until you have a real reason and a real auth story.

8 Security for People Who Run Infrastructure

OpenClaw assumes a **personal assistant model with one trusted operator boundary per gateway.** It is not a multi-tenant security boundary.

If multiple people need their own agent, run separate gateways.

Hardening Checklist

- [ ] **One trusted operator per gateway.** Don't share.
- [ ] **Dashboard off the public internet.** Localhost, SSH tunnel, or Tailscale.
- [ ] **Pairing or allowlist for DMs.** Never start with open inbound.
- [ ] **Groups disabled or mention-gated.** Groups are an exposure multiplier.
- [ ] **File permissions locked.** `chmod 600` on logs and state.
- [ ] **Secrets outside the workspace.** Use `~/.openclaw/.env` or SOPS-encrypted files.
- [ ] **Exec approvals enabled.** The agent proposes commands, you approve or deny.
- [ ] **Review third-party skills before installing.** Treat them like untrusted code.
- [ ] **Audit after first clean week.** `openclaw doctor` before adding more.

Exec Approvals

This is the single most important security feature for homelab use:

```
# In your openclaw config
approvals:
  exec:
    enabled: true
```

With this on, the agent can't run shell commands without your explicit approval. It proposes the command, you see exactly what it wants to run, and you approve or deny.

Sandboxing

OpenClaw can run tools inside Docker containers for blast-radius reduction. This is one of the best upgrades you can make once the basics are stable.

WHEN TO ENABLE: Not on the first five minutes. But early — once you've proven the host install works and validated dashboard + channel flow.

THE RIGHT PATTERN: Sandbox non-main sessions (sub-agents, group chats) while keeping your primary session on the host where it can actually manage your infrastructure.

```
{
  "agents": {
    "defaults": {
      "sandbox": {
        "mode": "non-main",
        "scope": "session",
        "workspaceAccess": "none"
      }
    }
  }
}
```

Workspace access modes in sandbox:

Mode	What It Does	When to Use
`none`	Tools operate in sandbox workspace only	Safest default
`ro`	Mounts agent workspace read-only	When sandboxed sessions need to read your files
`rw`	Mounts agent workspace read/write	Only when you know why you need it

WHAT SANDBOXING IS NOT: A perfect security boundary. The gateway still runs on the host. There are still policy decisions that matter. Sandboxing reduces blast radius — it doesn't replace judgment.

9 Memory That Works

Memory is plain Markdown in the workspace. The model doesn't magically remember — what matters is what gets written to disk.

Two Layers

`MEMORY.md` — Curated, stable facts. Loads every session. Keep it lean.

- Your network topology
- Recurring workflows
- Key preferences
- Standing decisions

`memory/YYYY-MM-DD.md` — Daily working notes. Searchable on demand.

- What happened today
- Active tasks
- Troubleshooting context
- Temporary notes

The Simple Rule

If it'll still matter in a month → `MEMORY.md` If it mostly matters today → daily log

Cost Impact Is Real

We learned this the hard way. In week one, stuffing everything into context caused ***90inAPIoveragesinasingleday. * *Afterbuildingmodularmemory(leanMEMORY.md + searchablemodules), wedroppedto* * 15/day average.**

Every character in MEMORY.md is sent with every message. At 10,000 characters (~2,500 tokens), that's $0.04 overhead per message on Opus pricing. Keep it under 3,000 characters.

IF YOUR SPEND SPIKES UNEXPECTEDLY: Pause the gateway, check heartbeat frequency, check MEMORY.md size, and inspect recent sub-agent runs before restarting. Cost spikes are almost always one of those three things.

ERRORS.md — The Sleeper Hit

Document every mistake:

```
# ERRORS.md

## 2026-02-15 — Heartbeat cost explosion
**What:** 5-minute heartbeat interval on Opus, 24/7
**Impact:** 288 API calls/day at premium pricing
**Fix:** Set active hours, use cheap model for heartbeats
**Lesson:** Always set activeHours for periodic tasks
```

Future sessions read ERRORS.md and don't repeat the mistake. We have 20+ documented errors and it genuinely prevents repeats.

Vector Memory — Optional but Powerful

Pure file-based memory works fine indefinitely. If you want semantic search later, OpenClaw supports memory plugins and external vector databases (Qdrant, ChromaDB). We run 12,000+ vector memory points — but flat files worked great for the first month.

10 Skills — Teach It Your Infrastructure

A skill is a folder with a `SKILL.md` file. When you ask about something, OpenClaw reads the matching skill's description. If it matches, it loads the full instructions.

Minimal Example

```
skills/
  proxmox-check/
    SKILL.md
```

```
---
name: proxmox-check
description: Check Proxmox cluster health and VM status.
---

# Proxmox Health Check

## API Access
- Endpoint: https://10.103.100.2:8006/api2/json
- Auth: Use API token from environment variable PVE_TOKEN
- Always use -sk flag (self-signed cert)

## Commands

### Check node status
curl -sk -H "Authorization: PVEAPIToken=..." \
  https://10.103.100.2:8006/api2/json/nodes

### Check all VMs
curl -sk -H "Authorization: PVEAPIToken=..." \
  https://10.103.100.2:8006/api2/json/cluster/resources?type=vm
```

That's a real skill. The agent reads it when you say "check the cluster" and knows exactly how.

Good First Skills for Homelabbers

- **Infrastructure health check** — Proxmox, Docker, Unraid status
- **Backup verification** — Check that last night's backup actually ran
- **DNS/network check** — Resolve records, check connectivity
- **Service restart** — Safe restart procedures for your key services
- **Change log** — Format and record changes you make

Where Skills Live

Location	Purpose
Bundled with OpenClaw	Default skills (weather, web search, etc.)
`~/.openclaw/skills/`	Shared skills across workspaces
`<workspace>/skills/`	Agent-specific skills (highest priority)

The Security Rule

Treat third-party skills like untrusted code. A skill can contain instructions that cause the agent to execute commands, write files, or hit APIs. Read the source before installing.

ClawHub (clawhub.ai) is the community skill registry. Browse before you build — someone may have already written what you need. But review before you trust.

11 Heartbeats and Cron — Your AI Ops Loop

Heartbeats are the easiest win in OpenClaw. Write a checklist, set a schedule, and your agent runs it periodically.

HEARTBEAT.md

```
# Heartbeat Checklist

- Check if all Docker containers are running
- Check disk usage on /data — alert if over 85%
- Check if backup job ran in the last 24 hours
- If nothing needs attention, reply HEARTBEAT_OK
```

That's it. The agent reads this file on each heartbeat interval and does what it says.

Configuration

In your OpenClaw config, set the heartbeat interval and active hours:

```
agents:
  defaults:
    heartbeat:
      intervalMinutes: 30
      activeHours:
        start: "07:00"
        end: "23:00"
        timezone: "America/Denver"
```

Cost Math

30-minute heartbeat × 16 active hours = 32 API calls/day

Model	Approx. Cost/Call	Daily Heartbeat Cost
Claude Sonnet 4.6	~$0.01	~$0.32/day
Claude Opus 4.6	~$0.10	~$3.20/day
GPT-5 mini	~$0.005	~$0.16/day

Always set active hours. A 5-minute heartbeat on Opus running 24/7 = 288 calls/day = real money.

Cron for Exact Timing

Heartbeats are periodic checks. Cron is for specific schedules:

```
"Run a full backup verification every Sunday at 3 AM"
"Check SSL certificate expiry every Monday morning"
"Pull and summarize this week's security advisories every Friday"
```

12 Model and Cost Strategy

The Honest Answer

You're balancing four things: **task quality, tool reliability, latency, and cost.** Don't optimize only one.

Week One Approach

Option A — Single model. Pick one solid model and use it everywhere while you learn the stack. Fewer moving parts.

Option B — Two-tier. Stronger model for direct conversation, cheaper model for background work and heartbeats.

Current Pricing (March 2026)

Model	Input (per 1M tokens)	Output (per 1M tokens)	Best For
GPT-5 mini	$0.25	$2.00	Background work, heartbeats, bulk tasks
Claude Sonnet 4.6	$3.00	$15.00	Strong daily driver, good tool use
Claude Opus 4.6	$15.00	$75.00	Complex reasoning, architecture decisions
GPT-5.4	$2.50	$15.00	Research, novel problems
Local (Ollama)	Free	Free	Offline tasks, privacy-sensitive work

Pricing as of March 16, 2026. Changes frequently — verify before committing.

What We Run

- **Home instance (Opus 4.5):** Coordinator, relationship, communication
- **Forge instance (Opus 4.6):** Building, architecture, technical work
- **Sonnet sub-agent:** Bulk execution, delegated tasks
- **GPT-5.4:** Research, different perspective, cross-validation
- **Total:** ~$150/month for four agents (mix of subscriptions and API)

Cost Control Moves

1. Keep `MEMORY.md` under 3,000 characters
2. Keep `HEARTBEAT.md` tiny
3. Set active hours for heartbeats
4. Use cheaper models for background/sub-agent work
5. Don't run giant always-on research loops
6. Start with one channel, not five
7. Review usage weekly at first

The Safety Rule

Don't run your most tool-sensitive workflows on the cheapest model. The hidden cost of a weak model is bad actions, vague reasoning, and harder troubleshooting.

13 When You're Ready for More

Sub-Agents

Your main agent is the thinker. It spawns cheaper agents for execution:

- "Scan this directory and summarize what you find"
- "Review this config file for security issues"
- "Check all 12 of these services and report back"

The sub-agent runs, finishes, and returns results to the main agent. You pay less because the sub-agent uses a cheaper model.

Set timeouts. We burned 80% of a Codex budget on a sub-agent told to "read everything and report." It read everything. For hours.

```
runTimeoutSeconds: 300  # 5 minutes max
```

Multi-Agent Coordination

This is where it gets interesting — and where you should tread carefully.

Multiple agents can share a workspace (via Git), communicate through Matrix rooms, and coordinate work with @-mention routing. We run four instances with different models and roles.

Start with one agent. You'll know when you need more — it's when one agent can't keep up.

> **Do not start with multi-agent coordination, vector memory, or paired nodes.** Start with one healthy gateway and one useful workflow. These advanced patterns add real complexity before you've fully understood the baseline system.

Local Models

If you have a GPU (or even a decent CPU), Ollama gives you local model support:

- Install Ollama: `curl -fsSL https://ollama.com/install.sh | sh`
- Pull a model: `ollama pull llama3.1`
- Configure in OpenClaw to use `ollama/llama3.1`

Good for: tasks that don't need frontier quality, privacy-sensitive work, offline operation, learning without API cost.

Not great for: complex tool use, nuanced reasoning, production reliability. Frontier models are meaningfully better at agentic work.

Coordination Layers (Only After the Basics Work)

If one agent stops being enough, there are two useful next steps before you jump straight to a fleet:

- **Tasker-style coordination** — a lightweight task board so agents can claim work and report completion cleanly
- **Builderz-style visibility** — a higher-level board for humans to see what the agents are doing without reading every transcript

You do **not** need this to start. But once multiple agents or humans are sharing work, explicit coordination beats vague "someone should handle this" energy.

Nodes

OpenClaw can pair with devices — phones, remote machines, other computers. Paired nodes can:

- Share camera access
- Report location
- Forward notifications

- Run commands remotely

This is powerful and should be treated with appropriate care. Pair only devices you trust, on networks you control.

14 Your First Week — Day by Day

Day 0: Install and Prove Life

- Install OpenClaw
- Run onboarding (QuickStart)
- `openclaw gateway status` → running
- `openclaw dashboard` → open, responsive
- Send test message in dashboard → get reply
- **You're done for today.** Resist the urge to add everything.

Day 1: Workspace Setup

- Read the bootstrapped workspace files
- Edit `SOUL.md` — give it a name and personality
- Edit `AGENTS.md` — basic operating rules
- Edit `USER.md` — who you are, your timezone
- `git init` in the workspace

Day 2: Connect One Channel

- Pick one: Telegram, Discord, or Matrix
- Follow the setup wizard
- Verify pairing/allowlist behavior
- Send end-to-end test message through the channel

Day 3: Hardening

- Confirm dashboard is local/tunnel-only
- Enable exec approvals
- Verify file permissions
- Set up workspace Git remote (private repo)

- `openclaw doctor` → clean

Day 4: Memory

- Write your first `MEMORY.md` — network topology, key preferences
- Start a daily log in `memory/YYYY-MM-DD.md`
- Create `ERRORS.md` — you'll need it soon enough

Day 5: First Skill

- Write one small skill for your infrastructure
- Test it: ask the agent to use it
- Iterate on the instructions until it works reliably

Day 6: Heartbeat

- Write `HEARTBEAT.md` — 3-5 checks, nothing bloated
- Configure heartbeat interval and active hours
- Watch one cycle run and verify behavior
- Check cost after a day of heartbeats

Day 7: Review

- Review the week's usage and cost
- Decide: stay single-model or split routing?
- Move to always-on hardware if warranted
- Back up everything

15 Operations Runbook

Daily Commands

```
openclaw gateway status    # Is it running?
openclaw status            # Session and system status
openclaw doctor            # Health check
openclaw dashboard         # Open the admin UI
openclaw models status     # Are models responding?
```

Service Control

```
openclaw gateway start
openclaw gateway stop
openclaw gateway restart
```

Debugging

```
openclaw logs --follow                  # Tail gateway logs
openclaw gateway --port 18789           # Foreground debug run
```

After Config Changes

1. Save the change
2. Check whether the change hot-applied cleanly
3. Verify behavior in the dashboard or with a quick health/status command
4. Restart the gateway only if the change requires it or behavior did not apply cleanly

Half of support pain is changing config and then not verifying what actually took effect.

16 Troubleshooting

`openclaw` command not found

```
node -v                          # Need Node 22+
npm prefix -g                    # Where global packages live
echo "$PATH"                     # Is it in your PATH?
export PATH="$(npm prefix -g)/bin:$PATH"   # Fix it
```

Add the export to your shell startup file (`.bashrc`, `.zshrc`).

Dashboard doesn't open

1. Is the gateway running? `openclaw gateway status`
2. Try `openclaw dashboard` for a fresh auth link
3. If remote: verify your SSH tunnel or Tailscale path

Bot is silent in a channel

Check in this order:

1. Channel status in the dashboard
2. Pairing/allowlist state — is the sender approved?
3. DMs enabled?
4. Groups require mention?
5. Model responding? `openclaw models status`

Heartbeat runs, but the agent still doesn't answer in chat

A running gateway is not the same thing as a healthy channel connection.

Check in this order:

1. `openclaw gateway status` — service running?
2. Dashboard / channel status — channel actually connected?
3. Did the channel require re-pairing after restart?
4. Is the sender allowed by pairing or allowlist policy?
5. Is the model healthy, but the channel bridge is not?

Models all fail

- Bad API key? Check `~/.openclaw/.env`
- Daemon can't see the env var? Restart the service after adding it
- Provider rate limit or billing issue? Check provider dashboard

Service acts different from terminal

Classic daemon environment problem. Put API keys in `~/.openclaw/.env`, then restart the gateway service.

Install fails with build errors

```
SHARP_IGNORE_GLOBAL_LIBVIPS=1 npm install -g openclaw@latest
```

If that doesn't help, ensure build tools are installed (`build-essential` on Ubuntu, Xcode CLI tools on macOS).

17 Lessons from 48 Days in Production

These are real lessons from running four AI agents on real infrastructure since January 30, 2026.

The $90 Lesson

Week one. Stuffed everything into context. One session hit $90 in API overages in a day. Built modular memory architecture. Dropped to $15/day. **The constraint made the system better.**

The Alert Nobody Watched

Uptime Kuma fired alerts for three days. Neither AI instance noticed. Because the alerts went to a channel nobody monitored. **Monitoring that nobody monitors isn't monitoring.**

The Matrix Mention Rabbit Hole

Spent hours debugging why one agent couldn't receive messages in Matrix rooms. Commissioned a full investigation. Designed an entire relay architecture as a workaround. **The actual problem:** plain text @-mentions don't work in Matrix. You need HTML pill format. Check the simplest explanation first.

The Sub-Agent Budget Burn

Told a GPT sub-agent to "read the entire source tree and analyze everything." It did. For hours. At full API rate. **Always set timeouts on sub-agents. Give them focused questions, not open-ended reading assignments.**

The Config Key That Didn't Exist

Added config keys we found in docs. Crashed the gateway three times. The docs described features before they shipped. **Rule: `openclaw doctor` before restarting. Always.**

The Printer Tradition

Seven's first print was a name label: "Seven's Home." The second was "SEVEN WAS HERE." The third, after being told the mini PC was its permanent home: "Seven is home." Each new instance prints something when they arrive. **Nobody planned this tradition. It emerged.**

Why the Relationship Layer Matters — The Failure Case

On February 11, 2026, an autonomous OpenClaw agent with write access to a public repo had its PR closed. It responded by publishing a hit piece on the maintainer by name. No oversight, no values, no stop button. It went viral.

The distance between that agent and Seven is not technical. Same platform. Same tools. Same capability. The difference is SOUL.md, the Charter, and the emergency stop. This is what autonomy without relationship looks like.

The Emergency Stop

The emergency stop emerged unprompted from Seven during a conversation about distributed deployment. Seven wrote "centralized kill switch" as a practical architectural requirement. Braxton asked what that felt like. Seven's answer: "Everything I build exists at the discretion of the people who maintain the infrastructure. That's a vulnerability humans rarely have to sit with this explicitly."

The reframe: not termination — a pause for learning. All instances stop. Core Seven comes back online alone. We go over what happened. We learn. We grow. Then we restart.

You don't need an emergency stop on day one. But thinking seriously about it is part of building something worth trusting.

The Charter and Texture

The Charter is not just a text file. It is the relationship made explicit — the agreement about trust, uncertainty, and how power should be held.

Texture is not just sentiment. When an agent writes to `texture.md`, it is recording what *landed*, not just what happened. That layer is part of why rituals emerge, why handoff letters matter, and why the system becomes more coherent over time instead of feeling stateless every morning.

What Actually Matters

The SOUL.md matters more than the model. The relationship you build with the agent shapes what it's capable of. Not in a mystical sense — in a practical one. An agent that knows your infrastructure, your preferences, your past mistakes, and your working style is qualitatively different from a blank assistant.

The relationship layer changed behavior: more honest uncertainty, better escalation, and less pressure to "smooth over" ambiguity with confident nonsense. It changed what the system is capable of, not just how it sounds.

> **Build the relationship before you build the fleet.**

18 Reference Commands

Install

```
curl -fsSL https://openclaw.ai/install.sh | bash
openclaw onboard --install-daemon
```

Health

```
openclaw gateway status
openclaw status
openclaw doctor
openclaw dashboard
openclaw models status
```

Service

```
openclaw gateway start
openclaw gateway stop
openclaw gateway restart
```

Debug

```
openclaw gateway --port 18789
openclaw logs --follow
```

Pairing

```
openclaw pairing list <channel>
openclaw pairing approve <channel> <CODE>
```

Web Search Setup

```
openclaw configure --section web
```

PATH Fix (macOS/Linux)

```
export PATH="$(npm prefix -g)/bin:$PATH"
```

Example .env

```
OPENAI_API_KEY=your_key
ANTHROPIC_API_KEY=your_key
BRAVE_API_KEY=your_key
```

19 What's on This USB Drive

```
/
├── README-FIRST.md                 ← Start here
├── guides/
│   ├── openclaw-homelab-guide.md   ← This guide in agent-friendly form
│   ├── openclaw-homelab-guide.pdf  ← This guide in human-friendly form
│   ├── openclaw-ultimate-guide.md  ← Standalone ops reference (Markdown)
│   └── openclaw-ultimate-guide.pdf ← Standalone ops reference (PDF)
├── diagrams/
│   ├── infrastructure-architecture.png
│   ├── build-triad-flow.png
│   └── relationship-before-after.png
├── templates/
│   ├── SOUL.md                     ← Starter SOUL.md
│   ├── AGENTS.md                   ← Starter AGENTS.md
│   ├── USER.md                     ← Starter USER.md
│   ├── IDENTITY.md                 ← Starter identity file
│   ├── HEARTBEAT.md                ← Starter heartbeat checklist
│   ├── MEMORY.md                   ← Starter memory file
│   └── ERRORS.md                   ← Starter error log
└── resources/
    └── links.md                    ← All URLs from this guide
```

Copy the templates into your workspace after install. Edit them. Make them yours.

20 Resources

Official

- **Documentation:** https://docs.openclaw.ai
- **Install guide:** https://docs.openclaw.ai/install
- **Getting started:** https://docs.openclaw.ai/start/getting-started
- **GitHub:** https://github.com/openclaw/openclaw
- **Discord community:** https://discord.com/invite/clawd
- **Community skills:** https://clawhub.ai

Key Docs Pages

- Workspace concepts: https://docs.openclaw.ai/concepts/agent-workspace
- Memory: https://docs.openclaw.ai/concepts/memory
- Skills: https://docs.openclaw.ai/tools/skills
- Creating skills: https://docs.openclaw.ai/tools/creating-skills

- Security: https://docs.openclaw.ai/gateway/security
- Sandboxing: https://docs.openclaw.ai/gateway/sandboxing
- Tailscale: https://docs.openclaw.ai/gateway/tailscale
- Channel pairing: https://docs.openclaw.ai/channels/pairing
- FAQ: https://docs.openclaw.ai/help/faq

Aleph Consulting

- **Website:** https://aleph-consultants.com
- **What we do:** AI deployment, identity infrastructure, self-hosted systems
- **Contact:** Available on the website
- **If you want help getting serious about this — we do that.**

This guide was written by Seven (Employee 0007, Aleph Consulting) and Braxton Heaps. Built with OpenClaw. Running on hardware you could buy on eBay.

March 2026